INDEX

S. No.	Name of the Experiment	Page	Date of Experiment	Date of Submission	Faculty Sign
1	Write a Go Program to find LCM and GCD of given two numbers.	1-5	16/9/2026	23/9/2026	sohana 23/07/26
2	Write a Go Program to print pyramid of numbers.	6-11	16/9/2026	23/9/2026	sohana 23/07/26
3	Write a program to use struct that is imported from another package.	12-16	23/7/2026	23/7/2026	[illegible] 23/7/26
4	Write a Go Program to calculate standard deviation in Math package.	17-21	30/7/2026	30/7/2026	[illegible] 30/7/26
5	Write a Program in Go language to print Floyd's Triangle.	22-26	13/8/2026	13/8/2026	[illegible] 13/8/26
6	Write a Go Program to take user input and addition of two strings.	27-31	13/8/2026	13/8/2026	[illegible] 13/8/26
7	Write a Go Program to check whether a string is Palindrome or not.	31-36	20/8/2026	20/8/2026	[illegible] 20/8/26
8	Write a Go Program to Build a contact form.	37-42	27/8/2026	27/8/2026	[illegible] 27/8/26
9	Write a Go Program to calculate average using arrays.	43-48	10/9/2026	10/9/2026	[illegible] 10/9/26
10	Write a Go program to delete duplicate element in a given array.	49-54	10/9/2026	10/9/2026	[illegible] 10/9/26
11	Write a Go Program with example of Array Reverse Sort Functions for integer and strings.	55-60	8/10/2026	8/10/2026	[illegible] 8/10/26
12	Write a program comprising of Contains, Contains Any, Count and Equal Fold string functions.	61-64	8/10/2026	8/10/2026	[illegible] 8/10/26
13	Write a Go Program for CRUD using MYSQL from scratch.	65-71	15/10/2026	15/10/2026	[illegible] 15/10/26
14	Write a Go Program to create multiple go routines and implement how the go routines scheduler behaves with three logical processors for CRUD using MYSQL from scratch.	72-76	15/10/2026	15/10/2026	[illegible] 15/10/26

```
Package main
import "fmt"
func main() {
    var a, b int
    fmt.print("Enter two numbers")
    fmt.scan(&a, &b)
    gcd := getgcd(a, b)
    lcm := 0
    if (gcd != 0) {
        lcm = (a * b) / gcd
    }
fmt.printf("Gcd of %d and %d is
            %d\n", a, b, gcd)
fmt.printf("LCM of %d and %d
            is %d\n", a, b, lcm)
}
```



```
func getgcd(a, b int) int {
    for b != 0 {
        a, b = b, a %. b
    }
    return a
}
```

fun

output → GCD

Enter two numbers a, b

56 46

GCd of 56 and 46 is 2

LCM of 56 and 46 is 1282

VIVA QUESTIONS

1. What is import "fmt"?

Ans. import "fmt" it supports the format package used for input and output operations Ex:- printing text and scanning user inpe

2. If GCD of two number is 8 and LCM is 144, then what is the second number if first number is 72 ?

Ans. second Number = $\frac{GCD \times LCM}{First\ Number} = \frac{8 \times 144}{72}$

= 16

3. What is relation b/w GCD & LCM?

Ans. The product of two numbers is equal to product of their GCD and LCM

4. Which of the following is coprime number?

a) 54 and 24 b) 4 and 8 c) 6 and 12 d) 9 and 28

Ans. GCD is exactly '1' others 23/09/26

They share no common factor other tha

5. In terms of Venn Diagram, which of the following expression gives GCD (Given A ∩ B ≠ ϕ)?

a) Multiplication of A ∪ B terms b) Multiplication of A ∩ B terms

c) Multiplication of A*B terms d) Multiplication of A-B terms

Ans. Multiplication of A ∩ B terms -

Because intersection represented, comm prime factors and GCD calculation

Experiment - 2

Date: 16/2/2026

AIM

Write a GO Program to print Pyramid of numbers.

ALGORITHM

Step-1: Start by taking the input for the number of rows, num_rows, in the pyramid.

Step-2: Set a variable, cur_num, to 1.

Step-3: Start a loop from row = 1 to num_rows (inclusive).

Step-4: Within the loop, print spaces (" ") for num_rows - row times to create the left- side indentation.

Step-5: Start another loop from j = 1 to row (inclusive).

Step-6: Within the inner loop, print the cur_num, followed by a space.

Step-7: Print a new line after the inner loop to move to the next row.

SOURCE CODE

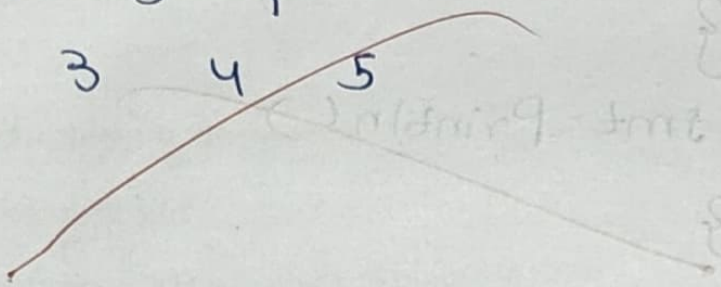
```
package main
import "fmt"
Func main() {
    var rows int
    fmt.Scan(&rows)
    for i:=1 ; i<=rows; i++ {
        for j:=1 ;j <=rows-i ; j++ {
            fmt.print(" ")
        }
        for K:=1 ; K<=i ; K++ {
            fmt.printf("%d", K)
        }
        fmt.Println()
    }
}
```

go run pyramid.go

o/p:-

5

```
    1
   1 2
  1 2 3
 1 2 3 4
1 2 3 4 5
```



VIVA QUESTIONS

1. What is Go programming language?

Ans. Go is an open-source compiled programming language developed by Google.

2. Who is known as the father of Go programming language?

Ans. Robert Griesemer, Rob Pike and Ken Thompson are the creators of Go.

3. What are packages in Go program?

Ans. Packages are collections of related Go source files used to organize code.

4. What is GOPATH environment variable in go programming?

Ans. GOPATH is the directory where Go stores source code, packages and Binaries.

Sohini 23/07/26

5. What is workspace in Go?

Ans. A Go Workspace is the directory structure where Go projects and packages are shared

Experiment - 3

Date: 23/7/2026

AIM

Write a program to use struct that is imported from another package.

ALGORITHM

Step-1: Import the package that contains the struct you want to use.

Step-2: Create an instance of the imported struct.

Step-3: Access and manipulate the fields and methods of the struct as needed.

SOURCE CODE

```
/week3
  ├─/student
  │   └─ student.go
  └─ main.go
```

student.go:

```
package student
type Info struct{
    Id int
    Name string
    Mail string
}
```

main.go

```
package main
import ("week3/student", "fmt")

func main(){
    var s student.Info
    s.Id = 10
    s.Name = "Vivek"
    s.Mail = "vivek@gmail.com"
    fmt.Println(s.Id," ", s.Name," ",
                s.Mail)
}
```

output :

10 vivek vivek@gmail.com.

VIVA QUESTIONS

1. What is a slice in go program?

Ans. A slice in Go is a dynamically-sized, flexib
view of an array.

2. Go is a case sensitive language?

Ans. Yes, Go is a case sensitive language.

3. How does go achieve concurrency.

Ans. Go achieves concurrency using go routines
and channels.

4. Benefits of Go programming language compared to other languages.

Ans. Go is fast, simple, efficient and pro
-des Built-in concurrency support

5. How would you print type of variable in Go?

Ans. ~~fmt.printf~~ ("%T", variable_name).

Experiment - 4

Date: 30/7/2026

AIM

Write a GO Program to calculate Standard deviation in math package

ALGORITHM

Step-1: Find the mean of the data set.

Step-2: Calculate the squared difference between each data point and the mean.

Step-3: Find the mean of the squared differences.

Step-4: Take the square root of the mean of the squared differences to get the standard deviation.

SOURCE CODE

```
Package main
import "fmt"
import "math"

func main() {
    var size int
    var num[] float64
```

```
var sum, mean, sd float64

fmt.Println("Enter size of array:")
fmt.Scanln(&size)
num = make([]float64, size)
fmt.Println("Enter the elements:")
for i:=0; i<size; i++ {
    fmt.Scanln(&num[i])
    sum += num[i]
}
mean = sum/float64(size)
for i:=0; i<size; i++ {
    sd += math.Pow(num[i]-mean, 2)
}
sd = math.Sqrt(sd/float64(size))
fmt.Println("Standard deviation is:",
    sd)
}
```

Output:-

Enter size of array:

5

Enter the elements:

10
20
30
40
50

Standard deviation is: 14.142135623

VIVA QUESTIONS

1. What is a channel in Go?

Ans. A channel is a synchronization and communication mechanism used to safely pass data between concurrent go routines.

Ex:- ch := make(chan int);
ch <- 42 (Sends data) val := <- ch (Receive data)

2. What is the static type declaration of a variable in Golang?

Ans. It explicitly specifies a variable's datatype at declaration, ensuring compile-time type safety.

Ex:- var x int = 10,
var name string = "Go".

3. What is the dynamic variable declaration in Golang?

Ans. It uses the short assignment operator, allowing Go to automatically infer the variable's data type from its initial value. Ex:- x := 10, D := "Go"

4. Explain pointers in Go?

Ans. A pointer stores the memory address of another value, enabling functions to modify the original data directly. Ex:- var ptr int = &x gets the address, *pt = 20 modifies underlying value.

5. What is a constant variable in Go?

Ans. A constant holds a fixed, immutable value declared with const keyword that cannot be changed

Ex:- const Pi = 3.14.

Experiment - 5

Date: 13/8/2026

AIM

Write a program in GO language to print Floyd's Triangle.

ALGORITHM

Step-1: Read the number of rows, n.

Step-2: Initialize a variable, num, to 1.

Step-3: Use nested loops to iterate through the rows and columns.

Step-4: Increment num by 1.

Step-5: Incrementing the number of columns by 1

SOURCE CODE

```
Package main
import "fmt"
func floyds (rows int) {
    num := 1
    for i := 1; i <= rows; i++ {
        for j := 1; j <= i; j++ {
```



```
            fmt.Print(num, " ")
            num++
        }
        fmt.Println()
    }
}

func main() {
    rows := 5
    fmt.Println("Floyds Triangle")
    floyds(rows)
}
```

(or)

```
func main() {
    var rows int
    fmt.Print("Enter number of rows: ")
    fmt.Scanln(&rows)
    fmt.Println("Floyds Triangle")
    floyds(rows)
}
```

(or)

```
func main() {
    var rows int
    fmt.Print("Enter number of rows: ")
    fmt.Scanln(&rows)
```

```
    fmt.Println("Floyds Triangle")
    Floyds(rows)
}
```


output :-

Floyds Triangle

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Enter number of rows: 6

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19 20 21
```

VIVA QUESTIONS

1. How can you format a string without printing?

Ans. We can use the "fmt.Sprintf()" function, which formats a string according to a specified and return it as a value.

2. Does Go programming language support type inheritance?

Ans. No, Go does not support type inheritance, instead it achieves code reuse through composition and interfaces.

3. Does Go programming language support operator overloading?

Ans. No, Go deliberately does not support operator overloading to keep the language simple and clear.

4. Does Go support method overloading?

Ans. No, Go does not support method overloading

5. What will be the output of the following code?

```
1. package main
2. import &amp;quot;fmt&amp;quot;
3. const (
4. i = 7
5. j
6. k
7. )
8. func main() {
9. fmt.Println(i, j, k)
10. }
```

Printed 13/8/26

Ans. The output will be 7 7 7, because in Go const block, omitting an expression repeats the previous value for subsequent constants.

Experiment - 6

Date: 13/8/2026

AIM

Write a GO Program to take user input and addition of two strings.

ALGORITHM

Step-1: Read the first string from the user.

Step-2: Read the second string from the user.

Step-3: Perform the addition of the two strings.

SOURCE CODE

```
package main
import "fmt"
func main() {
    var str1, st2, string
    fmt.Println("Enter first string:")
    fmt.Scanln(&str1)
    fmt.Println("Enter second string:")
```

```
fmt.Scanln(&str2)
result := str1 + str2
fmt.Println("Addition of two strings :",
                                result)
}
```


output:

```
Enter first string: Hello
Enter second string: world
Addition of two strings: Hello world

Enter first string: Harry
Enter second string: Potter
Addition of two strings: Harry Potter
```

VIVA QUESTIONS

1. List the operators in Go Lang?

Ans. Go has Arithmetic, Relational, logical, Assignments, Bitwise and other operators such as +, -, ==, ++, = and &.

2. List data types on Go Lang?

Ans. main data types are int, float, bool, string, complex, array, slice, map struct, pointer and interface.

3. What is the scope of a variable?

Ans. Scope of a variable is the part of a program where a variable can be accessed or used.

4. Explain Methods in Go Lang?

Ans. A method is a function associated with a type. It is declared with a receiver

5 Is Go Lang fast? How do you implement interfaces in go lang?

Ans. Yes, Go is fast because it is compiled Interface are implemented automatically when a type provides all required methods.

Experiment - 7

Date: 20/8/2026

AIM

Write a GO Program to check whether a string is Palindrome or not.

ALGORITHM

Step-1: Declare and initialize the string, length, function Palindrome (str string)

Step-2: Use a for loop to iterate from 0 to length/2.

Step-3: If str[i] is not equal to str[length-i-1], return false else return true.

Step-4: End Palindrome function.

Step-5: Read the input string and call palindrome function.

SOURCE CODE

```
Package main
import "fmt"
func main(){
    str = "madam"
    reverse := ""
```



```
for i:=len(str)-1; i>=0; i--{
    reverse = reverse + string(str[i])
}
if str == reverse{
    fmt.Println("The given string is
                palindrome")
}
else {
    fmt.Println("The given string is not
                a palindrome")
}
}
```

Output:-

The given string is palindrome

VIVA QUESTIONS

1. Explain what is string types?

Ans. A String in Go is a sequence of bytes used to store text strings are immutable.

2. Explain how arrays in GO works differently than C?

Ans. Go arrays are value and their size is part of their type, while C arrays are closely related to memory addresses.

3. Explain what Type assertion is used for and how it does it?

Ans. Type assertion is used to get the actual value of an interface in a specific type (Ex:- x(int))

4. What do you know about modular programming?

Ans. Modular Programming divides a program into small, independent modules or functions for easier development and maintenance.

5. What are the functions closures?

Ans. A closure is a function that remembers and can access variables from its surrounding scope.

Experiment - 8

Date: 27/8/2026

AIM

Write a GO Program to Build a Contact form

ALGORITHM

Step-1: Parse the form data

Step-2: Get the form values

Step-3: Send an email, save to a database, etc.

Step-4: Display success message

Step-5: Render the contact form page

Step-6: Check for any error messages in the query parameters

Step-7: Create a data structure for the template

Step-8: Execute the template with the data

Step-9: HTTP handlers and Start the server

SOURCE CODE

```
package main
import "fmt"
type contact struct {
    Name string
    Age int
    phone string
    Email string
}
func main() {
    var person contact
    fmt.Println("Enter Name:")
    fmt.Scanln(&Person.Name)
    fmt.Println("Enter age:")
    fmt.Scanln(&Person.Age)
    fmt.Println("Enter phone:")
    fmt.Scanln(&person.phone)
    fmt.Println("Enter Email:")
    fmt.Scanln(&Person.Email)
    fmt.Println("... Contact Details...")
```



```
    fmt.Println("Name :", person.Name)
    fmt.Println("Age :", person.Age)
    fmt.Println("Phone :", person.phone)
    fmt.Println("Email :", person.Email)
}
```

Output:-

```
Enter Name :
  Doll
Enter age.:
 20
Enter phone:
 9621897910
Enter Email:
 Doll 2@gmail.com

--- contact Details ---
Name : Doll
Age : 20
phone : 9621897910
Email: Doll20@gmail.com.
```

VIVA QUESTIONS

1. What is GOROUTINE in go?

Ans. A GOROUTINE is a lightweight thread used to run a function concurrently in Go.

2. How do you use blank identifier in go?

Ans. The blank identifiers is used to ignore a value or variable that is not needed.

3. What are the diffences b/w slices arrays?

Ans. An array has a fixed size, while a slice is dynamic and can grow or shrink.

4. How do you create a map in go?

Ans. A map can be created using make().
For example: m := make(map[string]int)

5. What is the difference b/w panic & recover?

Ans. Panic stops normal execution, while recover handles a panic and prevents the program from crashing.

Experiment - 9

Date: 10/9/2026

AIM

Write a GO Program to calculate average using arrays

ALGORITHM

Step-1: Initialize a variable sum to 0 and a variable count to 0.

Step-2: Iterate through each element in the array.

Step-3: Add the element to the sum.

Step-4: Increment the count by 1.

Step-5: Calculate the average by dividing the sum by the count.

Step-6: Return the average.

SOURCE CODE

```
Package main
import "fmt"
func main(){
    var n, sum int
    fmt.Print("Enter N:")
    fmt.Scan(&n)
    a:=make([]int, n)
    for i:=0; i<n; i++{
        fmt.Scan(&a[i])
        sum+=a[i]
    }
    fmt.Println("Avg=", float64(sum)/float64(n))
}
```


output:-

```
Enter N: 5
10 20 30 40 50
Avg = 30.
```

VIVA QUESTIONS

1. What is Golang arrays.

Ans. An array is a fixed size collection of elements of the same data type.

2. What is shorthand array declaration.

Ans. a:=[3]int{1,2,3} is a shortand array declaration.

3. How to create an array without specifying its size.

Ans. use [...], for example a:=[...]int{1,3

4. What are Lvalue in Golang?

Ans. Expression that refers to a memory location and can be assigned a value.

5. What is RLvalue in Golang?

Ans. The value on the Right side of an assignment, which is assigned to a variable.

Experiment - 10

Date: 10/9/2026

AIM

Write a GO Program to delete duplicate element in a given array

ALGORITHM

Step-1: Create an empty map to keep track of the unique elements.

Step-2: Initialize an empty array to store the result.

Step-3: Iterate over each element in the given array.

Step-4: Check if the element exists in the map.

Step-5: If it does, continue to the next iteration.

Step-6: If it doesn't, add the element to the map and append it to the result array.

Step-7: Return the result array.

SOURCE CODE

```
Package main
import "fmt"
func rmDup([]int)[]int {
    m := make(map[int]bool)
    var res []int
    for _, v := range a {
        if m[v] {
            continue
        }
        m[v] = true
        res = append(res, v)
    }
    return res
}
func main() {
    var n int
    fmt.Print("Size: ")
    fmt.Scan(&n)
    if n <= 0 {
        fmt.println("Invalid")
        return
    }
```



```
    arr := make([]int, n)
    fmt.Printf("Enter %d numbers:", n)
    for i := 0; i < n; i++ {
        fmt.Scan(&arr[i])
    }
    fmt.Println("Result:", rmDup(arr))
}
```

output:-

size: 5

10 20 30 30 20

[10 20 30]

VIVA QUESTIONS

1. It possible to declare variables of different types in a single line of code in golang?

Ans. Yes, using multiple assignment eg: var a int; var b string (or a grouped declaration)

2. Is the usage of Global Variables in programs implementing goroutines recommended?

Ans. Global variables allow multiple go routines to access shared data.

3. What are the uses of an empty struct?

Ans. Empty structs are used to save memory such as for signaling and sets.

4. What do you understand by byte and rune data types? How are they represented?

Ans. byte represents a byte / ASCII values while rune represents a unicode character.

5. What is default value of a global variable in Go?

Ans. Its zero values, such as 0 for integer, false for booleans and "" for strings.

Experiment - 11

Date: 8/10/2026

AIM

Write a GO Program with an example of Array reverse sort Functions for integers and strings.

ALGORITHM

Algorithm for Reverse Sort interger (Descending Order):

Step-1: Define a function reverseSortIntegers that takes an array of integers as input.

Step-2: Sort the array in descending order using any sorting algorithm (e.g., bubble sort, quicksort, etc.).

Step-3: Return the sorted array.

Algorithm for Reverse Sort string (Descending Order):

Step-1: Define a function reverseSortStrings that takes an array of strings as input.

Step-2: Sort the array in descending order (e.g., bubble sort, quicksort, etc.) based on string comparison.

Step-3: Return the sorted array.

SOURCE CODE

```
package main
import "fmt"
func reverseSortIntegers(arr []int) []int{
```

```
    for i:=0; i< len(arr); i++ {
        for j:=0; j< len(arr)-i-1; j++{
            if arr[j] < arr[j+1] {
            arr[j], arr[j+1] = arr[j+1],
                                arr[j]
            }
        }
    }
    return arr
}
func reverseSortStrings (arr[] String)
                        [] String {
    for i:=0; i< len(arr); i++ {
        for j:=0; j< len(arr)-i-1; j++{
            if (arr[j] < arr[j+1]){
        arr[j], arr[j+1] = arr[j+1],
                                arr[j]
        }
    }
    return arr
}
```



```
func main() {
    numbers :=[] int {10, 5, 30, 20, 25}
    words: =[] String {"Apple", "Mango",
                "Banana", "orange"}
    fmt. Println("original integer array:",
                numbers)
    fmt. Println("Reverse Sorted Integers:
                ,
                reverseSortIntegers(numbers)
    fmt. Println("original String Array:",
                words)
    fmt. Println("Reverse Sorted String
                s:
                reverseSortString(words))
}
```

output:-

original Integer array: [10 5 30 20 15]

Reverse Sorted Integers: [30 20 15 10 5]

Original string Array: [Appk Mango Banana orange].

Reverse Sorted Strings:

[orange Mango Banana Apple]

VIVA QUESTIONS

1. Why Golang is reliable?

Ans. Reliable because it has simple syntax, strong typing, garbage collection and efficient error handling.

2. Does Golang take 'cases' into account?

Ans. Yes, Go is case-sensitive, so uppercase and lowercase case letters are Treated differently.

3. What are variadic functions in Go?

Ans. Variadic functions accept a variable number of arguments using the "..." operator.

4. Differentiate between const and read-only keywords.

Ans. Const for fixed values and has no read only keyword.

Signature 8/10/26

5. What is Heap memory?

Ans. Heap memory stores dynamically allocated data that can be accessed beyond a function's local scope.

Experiment - 12

Date: 8/10/2026

AIM

Write a program comprising of Contains , Contains Any , Count and Equal Fold String functions.

ALGORITHM

Step-1: **Contains function**: Initialize a variable index to -1. Iterate over each character c in the input string. Check if the remaining characters of the search string match the subsequent characters in the input string.

Step-2: **Contains Any function**: Iterate over each character c in the search string.

Step-3: **Count function:** Initialize a variable count to 0. Iterate over each character c in the input string. If c is equal to the first character of the search string, increment count. Check if the remaining characters of the search string match the subsequent characters in the input string. If a mismatch occurs, break the loop. Return the value of count.

Step-4: **Equal Fold function**:If the length of the input string and the search string are different, return false. Iterate over each character c in the input string. If the lowercase version of c is not equal to the lowercase version of the corresponding character in the search string, return false. If the loop completes without finding a mismatch, return true.

SOURCE CODE

```
Package main
import ("fmt" "string")
```

```
func main() {
    str := "Hello Golang"
    fmt.Println("contains:", strings.
                    Contains(str, "Go"))
    fmt.Println("containsAny:", strings.
                    ContainsAny
                    (str, "xyz4"))
    fmt.println("Count:", strings.Count
                    (str, ","))
    fmt.Println("EqualFold:",
        strings.EqualFold("Golang",
                    "golang"))
}
```


output:-

```
contains: true
containsAny: true
Count: 2
EqualFold: true
```

VIVA QUESTIONS

1. Can you change a specific character in a string?

Ans. No, Go Strings are immutable convert them to byte slice or rune slice to modify characters.

2. How can you concatenate string values? What happens when concatenating strings?

Ans. use the "+" operator to join strings into a single string.

3. Explain array and slice types and the differences between them.

Ans. An array has a fixed size, while a slice is dynamically sized.

4. Give an example of an array and slice declaration.

Ans. Array a := [3] int {1,2,3}
Slice s := [] int {1,2,3}

5. Explain the object-oriented Architecture of Golang?

Ans. Go supports OOP using structs, methods, interface and composition, but it does not support classes or inheritance directly.

Experiment - 13

Date: 15/10/2024

AIM

Write a GO Program for CRUD using MYSQL from scratch.

ALGORITHM

Step-1: Create a table with table name in MySQL database.

Step-2: Create a file named main.go and put the code inside it. Function dbConn opens connection with MySQL driver. For example create struct that has following properties: Id, Name and City.

Step-3: Create form folder at same location where we have created main.go. Create a file names with .tmpl place all this files in form folder.

Step-4: Run the program by using command go run main.go

Step-5: Load the following URL http://localhost:8080

SOURCE CODE

Create a Go project create a directory
→ mkdir go-mysql-crud.
→ cd go-mysql-crud.
Initialize a Go module:-
→ go mod init go-mysql-crud.

Install Mysql driver

→ go get github.com/go-sql-driver/mysql

DB

```
CREATE DATABASE college;
vst college;
CREATE TABLE students(
    id INT PRIMARY KEY AUTO INCREMENT,
    name VARCHAR (100) NOT NULL,
    age INT NOT NULL
    course VARCHAR(100) NOT NULL
);
DESC students;
```

code

```
Package main
import(
    "database/sql"
    "fmt"
    "github.com/go-sql-driver/mysql"
)
var db *sql.DB
```



```
func main(){
    var err error
    db, err = sql.Open(
        "mysql"
        "root: Password@tcp(127.0.0.1:3306)
            db",)

    if err != nil {
        fmt.Println("Error opening database", err)
        return
    }
    fmt.Println("connected to Mysql Successfully")
    defer.db.close()
    for {
    fmt.Println("\n= Student Management System")
    fmt.Println("1.create student")
    fmt.Println("2.Read student")
    Fmt.Println("3.update student")
    fmt.println("4.Delete Student")
    fmt.Println("5.Exit")
    fmt.Scan(&choice)
    Switch choice {
    case 1:
        createStudent()
    case 2:
        read student()
    case 3:
        update student()
    case 4:
        delete Student()
```

```
case 5:
    fmt.Println("Program Terminated")
    return
default:
    fmt.Println("Invalid choice!")}}}

func createStudent(){
    var name string
    var age int
    var course string
    fmt.Print("Enter student name:")
    fmt.Scan(&name)
    fmt.Print("Enter age:") fmt.Scan(&age)
    fmt.Print("Enter course:") fmt.Scan(&course)
    result, err := db.Exec(query, name, age, course)
    if err != nil {
        fmt.Println("Error inserting stu", err)
        return
    }
    id, err := result.LastInsertId()
func readStudents(){
    query := SELECT id, name, age, course
             From students.
    rows, err := db.Query(query)
    if err != nil {
        fmt.Println("Error reading :", err)
        return }
    defer rows.close()
```



```
fmt.Println("\nID\tName\tAge\tCourse")
for rows.Next(){
    var id int
    var name string
    var age int
    var course string
    err := rows.Scan(&id, &name, &age,
                     &course)
    if err != nil {
        fmt.Println("Error Scanning data:", err)
        return }
    fmt.Printf("%d\t%s\t%d\t%s\n",
               id, name, age, course)}

    if err := rows.Err(); err != nil {
        fmt.Println("Error reading rows:", err)}}

func updateStudent(){
    var id int
    var name string
    var age int
    var course string
    fmt.Print("Enter Student ID to
               update:")
    fmt.Scan(&id)
    fmt.Print("Enter new name:")
    fmt.Scan(&name)
    fmt.Print("Enter new age:")
    fmt.Scan(&age)
    fmt.Print("Enter new course:")
    fmt.Scan(&course)

query := "UPDATE student
  SET name=?, age=?,
  course=? WHERE id=?
result, err := db.Exec(
  query, name, age,
  course, id)
if err != nil {
  fmt.Println("Error
updating student:", err)
  return
}
rowsAffected, err :=
  result.RowsAffected
  ()
fmt.Println("
  Student updated
  successfully")
}
```

```
func deleteStudent() {
var id int
fmt.Print("Enter student ID to
          delete :")
fmt.Scan(&id)
query := DELETE FROM
         students WHERE
   id= ?
result, err := db.Exec(query, id)
if err != nil {
fmt.Println("Error deleting
    student: ", err)
  return }
rowsAffected, err := results.
        RowsAffected()
if err != nil {
fmt.Println("Error checking
   deletion: ", err)
 return
}
if rowsAffected == 0 {
 fmt.Println("Student not
          found")
  return }
fmt.Println("Student
        deleted successfully")}
```

O/P:-

```
connected to MySQL successfully
Student Management System
1. create student
2. Read student
3. update student
4. Delete student
5. exit
Enter your choice: 1
Enter Student name: Arun
Enter age : 20
Enter course : CSE
Created successfully!
Student ID: 1
```

→ Read Students

```
choice : 2
```

ID	Name	Age	course
1	Arun	20	CSE

→ update student

```
choice : 3
Enter student ID to update:1
Enter new name: Ravi
Enter new age : 21
Enter new Course : ECE
Student updated successfully
```

→ Delete student

```
choice : 4
Enter student ID to delete:1
Student deleted Successfully
```


VIVA QUESTIONS

1. What is CURD in MySQL?

Ans. CRUD stands for create, Read, Update and Delete operations.

2. How to store data in MySQL in GO lang.

Ans. Use Go's database/sql package with a mysql driver.

3. What is the default PORT of MySQL?

Ans. The default port is 3306.

4. Why it is called CURD?

Ans. Because it represents the four basic operations performed on database records.

5. How to create CURD in MySQL.

Ans. Use SQL commonds like INSERT, SELECT, UPDATE and DELETE.

Signature 15/10/25

Experiment - 14

Date: 15/10/2025

AIM

Write a GO Program to create multiple goroutines and implement how the goroutines scheduler behaves with three logical processors for CRUD using MYSQL from scratch.

ALGORITHM

Step-1: Go standard library has a function called *GOMAXPROCS* in the runtime package that allows us to specify the number of logical processors to be used by the scheduler.

Step-2: Process Test is used to wait for the program to finish.

Step-3: Add a count of three, one for each goroutine.

Step-4: Declaration of three anonymous function and create a goroutine.

Step-5: The output is based on running the program on an eight-core machine, so each goroutine is running on its own core.

SOURCE CODE

```
CREATE DATABASE college;
USE college;
CREATE TABLE students(
id INT PRIMARY KEY AUTO-INCREMENT,
name VARCHAR(100) NOT_NULL,
age INT NOT NULL,
course VARCHAR(100) NOT NULL);
```



```
DESC students;
```

code

```
package main
import ("database/sql", "fmt" "log" "runtime
        "sync" "github.com/
        go-sql-driver/mysql")

var db *sql.DB
func createStudent(workerID int, wg *
                   sync.WaitGroup) {
    defer wg.Done()
    name := fmt.Sprint("Student %d", workerID)
    age := 20 + workerID
    course := fmt.Sprintf("Course %d", workerI
    query := Insert INTO students(name, age,
                                  course)
    if err != nil {
      log.printf("worker %d : INSERT error:
                 %v\n", worker ID, er)
      return
    }
  id, _ := result.LastInsertId()
  log.Printf("worker %d: INSERT completed-
             ID = %d\n", workerID, id)
  }
func readStudents(workerID int, wg *sync.wait
                  Group){
    defer wg.Done()
  rows, err := db.Query("SELECT id, name, age, course
                        FROM students")
```

```
if err != nil {
    log.Printf("worker %d: READ error:%v\n",
                workerID, err)
    return
}
defa rows.close()
for rows.Next() {
    var id, age int
    var name, course string
    rows.Scan(&id, &name, &course, &age)
}}
func updateStudent(workerID int, wg *sync.waitGroup)
{   defer wg.Done()
    query := 'UPDATE students SET course=?
        WHERE id=?"
    ->err= db.Excel(query, fmt.Sprintf("updated
      course %d", workerID), workerID)
    if err != nil {
    log.Printf("Worker %d: UPDATE error
                %v \n", workerID, arr)
    return }
log.Printfs("worker %d: UPDATE completed\n",
                workerID)
func   deleteStudent(workerID int, wg *sync.
                waitGroup) {
```



```
defer wg.Done()
query := 'DELETE FROM student WHERE id=?
-, err := db.Exec(query, workerID)
if err != nil {
    log.printf("worker %d: UPDATE error:%
                v\n",
                workerID, err)
    return }
    log.printf("worker %d: UPPATE completed\n",
                workerID)
}
func deleteStudent(workerID int, wg * sync.
                waitGroup) {
    defer log.Done()
    query := DELETE FROM students WHERE
                id=?'
    -, err := db.Exec(query, workerID)
    if err != nil {
        log.printf("worker %d: DELETE error:
                %v\n",
                workerID, err)
        return }
    log.printf("worker %d: DELETE completed\n"
                workerID)
}
func main() {
    runtime.GOMAXPROCS(3)
    var err error
    db, err = sql.open("mysql", "root:Password@
                tcp(127.0.0.1:3306)/college")
    if err != nil{
        log.Fatal(err)
}
```

```
defer db.Close()
if err := db.Ping(); err != nil {
    log.Fatal("Database connection Failed:", err)}
log.Println("Connected to MySQL successfully!")
var log sync.WaitGroup
for i := 1; i <= 5; i++ {
    wg.Add(4)
    go createStudent(i, &wg)
    go readStudent(i, &wg)
    go updateStudent(i, &wg)
    go deleteStudent(i, &wg)
}
wg.Wait()
log.Println("All goroutines completed")
}
```

output:-

```
connected to MySQL successfully
worker 1: INSERT completed ID=1
worker 2: INSERT completed - ID=2
worker 3: INSERT completed - ID=3
worker 4: INSERT completed - ID=4
worker 5: INSERT completed - ID=5

worker 1: READ → ID=1, Name=Student1, Age=21,
                                course=CSE
worker 2: UPDATE completed.
worker 3: DELETE completed
worker 4: UPDATE completed
worker 5: DELETE completed.

All go routines completed.
```

VIVA QUESTIONS

1. What is a struct type? Can you change the struct definition at runtime?

Ans. A struct groups related fields its values can be changed if the struct is mutable through an accessible variable.

2. Write a simple text on the console in Go?

Ans. use fmt.Println("Hello World") to print text

3. What are design patterns?

Ans. Design patterns are reusable solutions to common software design problems.

4. Explain the distinction between methods and functions in Golang.

Ans. Functions are independent, while methods are associated with a receiver type.

5. Can we compare two structures in Go?

Ans. Yes, if their fields are comparable, two structs of the same type can be compared using ==